INTERFACE

# Customize your status line

Configure a custom status bar to monitor context window usage, costs, and git status in Claude Code

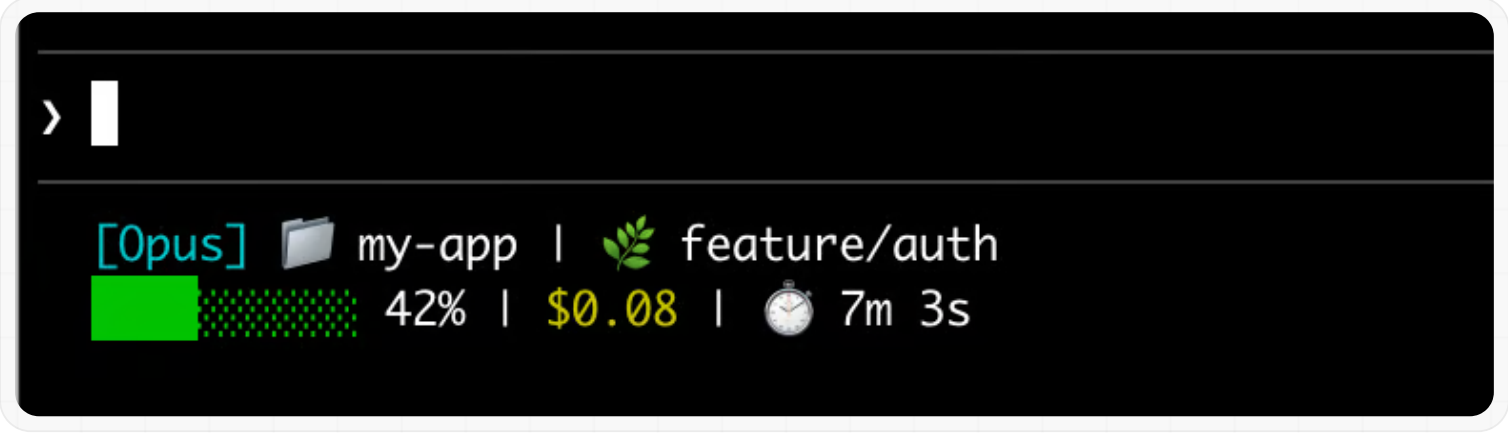
The status line is a customizable bar at the bottom of Claude Code that runs any shell script you configure. It receives JSON session data on stdin and displays whatever your script prints, giving you a persistent, at-a-glance view of context usage, costs, git status, or anything else you want to track.

Status lines are useful when you:

- Want to monitor context window usage as you work
- Need to track session costs
- Work across multiple sessions and need to distinguish them
- Want git branch and status always visible

The status line renders in its own row above the built-in footer badges and does not replace them. To add clickable link badges to the footer when an ID appears in the conversation, without writing a script, configure `footerLinksRegexes` instead.

Here's an example of a **multi-line status line** that displays git info on the first line and a color-coded context bar on the second.




This page walks through **setting up a basic status line**, explains **how the data flows** from Claude Code to your script, lists **all the fields you can display**, and provides **ready-to-use examples** for common patterns like git status, cost tracking, and progress bars.

# Set up a status line

Use the `/statusline` **command** to have Claude Code generate a script for you, or **manually create a script** and add it to your settings.

## Use the /statusline command

The `/statusline` command accepts natural language instructions describing what you want displayed. Claude Code generates a script file in `~/.claude/` and updates your settings automatically:

```
/statusline show model name and context percentage with a
progress bar
```

## Manually configure a status line

Add a `statusLine` field to your user settings (`~/.claude/settings.json`, where `~` is your home directory) or **project settings**. Set `type` to `"command"` and point `command` to a script path or an inline shell command. For a full walkthrough of creating a script, see **Build a status line step by step**.

```
{
  "statusLine": {
    "type": "command",
    "command": "~/.claude/statusline.sh",
    "padding": 2
  }
}
```

The `command` field runs in a shell, so you can also use inline commands instead of a script file. This example uses `jq` to parse the JSON input and display the model name and context percentage:

```
{
  "statusLine": {
    "type": "command",
    "command": "jq -r '\"[\\(.model.display_name)] \\
(.context_window.used_percentage // 0)% context\"'"
  }
}
```

The optional `padding` field adds extra horizontal spacing (in characters) to the status line content. Defaults to `0`. This padding is in addition to the interface's built-in spacing, so it controls relative indentation rather than absolute distance from the terminal edge.

The optional `refreshInterval` field re-runs your command every N seconds in addition to the **event-driven updates**. The minimum is `1`. Set this when your status line shows time-based data such as a clock, or when background subagents change git state while the main session is idle. Leave it unset to run only on events.

The optional `hideVimModeIndicator` field suppresses the built-in `-- INSERT --` text below the prompt. Set this to `true` when your script renders `vim.mode` itself, so the mode is not shown twice.

### Disable the status line

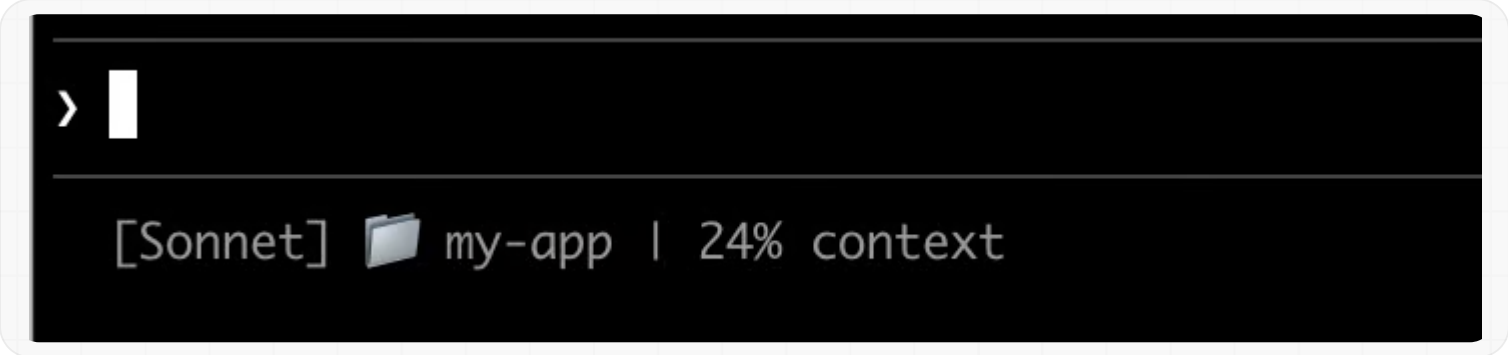
Run `/statusline` and ask it to remove or clear your status line (e.g., `/statusline delete`, `/statusline clear`, `/statusline remove it`). You can also manually delete the `statusLine` field from your settings.json.

## Build a status line step by step

This walkthrough shows what's happening under the hood by manually creating a status line that displays the current model, working directory, and context window usage percentage.

> Running `/statusline` with a description of what you want configures all of this for you automatically.

These examples use Bash scripts, which work on macOS and Linux. On Windows, see **Windows configuration** for PowerShell and Git Bash examples.




**1 Create a script that reads JSON and prints output**

Claude Code sends JSON data to your script via stdin. This script uses `jq`, a command-line JSON parser you may need to install, to extract the model name, directory, and context percentage, then prints a formatted line.

Save this to `~/.claude/statusline.sh` (where `~` is your home directory, such as `/Users/username` on macOS or `/home/username` on Linux):

```
#!/bin/bash
# Read JSON data that Claude Code sends to stdin
input=$(cat)

# Extract fields using jq
MODEL=$(echo "$input" | jq -r '.model.display_name')
DIR=$(echo "$input" | jq -r '.workspace.current_dir')
# The "// 0" provides a fallback if the field is null
PCT=$(echo "$input" | jq -r '.context_window.used_percentage
// 0' | cut -d. -f1)

# Output the status line - ${DIR##*/} extracts just the
folder name
echo "[$MODEL] 📁 ${DIR##*/} | ${PCT}% context"
```

### 2 Make it executable

Mark the script as executable so your shell can run it:

```
chmod +x ~/.claude/statusline.sh
```

### 3 Add to settings

Tell Claude Code to run your script as the status line. Add this configuration to `~/.claude/settings.json`, which sets `type` to `"command"` (meaning “run this shell command”) and points `command` to your script:

```
{
  "statusLine": {
    "type": "command",
    "command": "~/.claude/statusline.sh"
  }
}
```

Your status line appears at the bottom of the interface. Settings reload automatically, but changes won't appear until your next interaction with Claude Code.

## How status lines work

Claude Code runs your script and pipes **JSON session data** to it via stdin. Your script reads the JSON, extracts what it needs, and prints text to stdout. Claude Code displays whatever your script prints.

### When it updates

Your script runs after each new assistant message, after `/compact` finishes, when the permission mode changes, or when vim mode toggles. Updates are debounced at 300ms, meaning rapid changes batch together and your script runs once things settle. If a new update triggers while your script is still running, the in-flight execution is cancelled. If you edit your script, the changes won't appear until your next interaction with Claude Code triggers an update.

These triggers can go quiet when the main session is idle, for example while a coordinator waits on background subagents. To keep time-based or externally-sourced segments current during idle periods, set `refreshInterval` to also re-run the command on a fixed timer.

### What your script can output

- **Multiple lines**: each `echo` or `print` statement displays as a separate row. See the **multi-line example**.
- **Colors**: use **ANSI escape codes** like `\033[32m` for green (terminal must support them). See the **git status example**.
- **Links**: use **OSC 8 escape sequences** to make text clickable (Cmd+click on macOS, Ctrl+click on Windows/Linux). Requires a terminal that supports hyperlinks like iTerm2, Kitty, or WezTerm. See the **clickable links example**.

### Sizing output to the terminal

Claude Code captures your script's output instead of connecting it directly to the terminal, so `tput cols` and language-level width detection cannot read the terminal size from inside the script. Read the `COLUMNS` and `LINES` environment variables instead. Claude Code sets these to the current terminal dimensions before running your script. Requires Claude Code v2.1.153 or later.

> The status line runs locally and does not consume API tokens. It temporarily hides during certain UI interactions, including autocomplete suggestions, the help menu, and permission prompts.

## Available data

Claude Code sends the following JSON fields to your script via stdin:

| Field | Description |
| --- | --- |
| `model.id`, `model.display_name` | Current model identifier and display name |
| `cwd`, `workspace.current_dir` | Current working directory. Both fields contain the same value; `workspace.current_dir` is preferred for consistency with `workspace.project_dir`. |
| `workspace.project_dir` | Directory where Claude Code was launched, which may differ from `cwd` if the working directory changes during a session |
| `workspace.added_dirs` | Additional directories added via `/add-dir` or `--add-dir`. Empty array if none have been added |
| `workspace.git_worktree` | Git worktree name when the current directory is inside a linked worktree created with `git worktree add`. Absent in the main working tree. Populated for any git worktree, unlike `worktree.*` which applies only to `--worktree` sessions |
| `workspace.repo.host`, `workspace.repo.owner`, `workspace.repo.name` | Repository identity parsed from the `origin` remote, for example `"github.com"`, `"anthropics"`, `"claude-code"`. Absent outside a git repository or when no `origin` remote is configured |
| `cost.total_cost_usd` | Estimated session cost in USD, computed client-side. May differ from your actual bill |
| `cost.total_duration_ms` | Total wall-clock time since the session started, in milliseconds |
| `cost.total_api_duration_ms` | Total time spent waiting for API responses in milliseconds |
| `cost.total_lines_added`, `cost.total_lines_removed` | Lines of code changed |
| `context_window.total_input_tokens`, | Token counts currently in the context window, from the most recent API response. Input includes cache reads and writes. Before v2.1.132 these were |

| Field | Description |
| --- | --- |
| `context_window.total_output_tokens` | cumulative session totals |
| `context_window.context_window_size` | Maximum context window size in tokens. 200000 by default, or 1000000 for models with extended context. |
| `context_window.used_percentage` | Pre-calculated percentage of context window used |
| `context_window.remaining_percentage` | Pre-calculated percentage of context window remaining |
| `context_window.current_usage` | Token counts from the last API call, described in **context window fields** |
| `exceeds_200k_tokens` | Whether the total token count (input, cache, and output tokens combined) from the most recent API response exceeds 200k. This is a fixed threshold regardless of actual context window size. |
| `effort.level` | Current reasoning effort ( `low` , `medium` , `high` , `xhigh` , or `max` ). Reflects the live session value, including mid-session `/effort` changes. Ultracode is not a distinct level and reports as `xhigh` . Absent when the current model does not support the effort parameter |
| `thinking.enabled` | Whether extended thinking is enabled for the session |
| `rate_limits.five_hour.used_percentage` , `rate_limits.seven_day.used_percentage` | Percentage of the 5-hour or 7-day rate limit consumed, from 0 to 100 |
| `rate_limits.five_hour.resets_at` , `rate_limits.seven_day.resets_at` | Unix epoch seconds when the 5-hour or 7-day rate limit window resets |
| `session_id` | Unique session identifier |
| `session_name` | Custom session name set with the `--name` flag or `/rename` . Absent if no custom name has been set |
| `transcript_path` | Path to conversation transcript file |
| `version` | Claude Code version |
| `output_style.name` | Name of the current output style |
| `vim.mode` | Current vim mode ( `NORMAL` , `INSERT` , `VISUAL` , or `VISUAL LINE` ) when **vim mode** is enabled |
| `agent.name` | Agent name when running with the `--agent` flag or agent settings configured |
| `pr.number` , `pr.url` | Open pull request for the current branch. Mirrors the PR badge in the bottom status bar. Absent until a PR is found, when not in a git repository, or once the PR merges or closes |

| Field | Description |
| --- | --- |
| `pr.review_state` | Review status of the open PR: `approved`, `pending`, `changes_requested`, or `draft`. May be independently absent even when `pr` is present |
| `worktree.name` | Name of the active worktree. Present only during `--worktree` sessions |
| `worktree.path` | Absolute path to the worktree directory |
| `worktree.branch` | Git branch name for the worktree (for example, `"worktree-my-feature"`). Absent for hook-based worktrees |
| `worktree.original_cwd` | The directory Claude was in before entering the worktree |
| `worktree.original_branch` | Git branch checked out before entering the worktree. Absent for hook-based worktrees |

▸ Full JSON schema

## Context window fields

The `context_window` object describes the live context window from the most recent API response. As of v2.1.132, `total_input_tokens` and `total_output_tokens` reflect current context usage, not cumulative session totals.

- **Combined totals** (`total_input_tokens`, `total_output_tokens`): tokens currently in the context window. `total_input_tokens` is the sum of `input_tokens`, `cache_creation_input_tokens`, and `cache_read_input_tokens`; `total_output_tokens` is the output tokens from the most recent response. Both are `0` before the first API response.
- **Per-component usage** (`current_usage`): the same token counts broken out by category. Use this when you need cache hits separate from fresh input.

The `current_usage` object contains:

- `input_tokens`: input tokens in current context
- `output_tokens`: output tokens generated
- `cache_creation_input_tokens`: tokens written to cache
- `cache_read_input_tokens`: tokens read from cache

For what the cache fields mean and how they're billed, see **check cache performance**.

The `used_percentage` field is calculated from input tokens only: `input_tokens +`

`cache_creation_input_tokens + cache_read_input_tokens`. It does not include `output_tokens`.

If you calculate context percentage manually from `current_usage`, use the same input-only formula to match `used_percentage`.

The `current_usage` object is `null` before the first API call in a session, and again immediately after `/compact` until the next API call repopulates it.

## Examples

These examples show common status line patterns. To use any example:

1. Save the script to a file like `~/.claude/statusline.sh` (or `.py` / `.js`)
2. Make it executable: `chmod +x ~/.claude/statusline.sh`
3. Add the path to your **settings**

The Bash examples use `jq` to parse JSON. Python and Node.js have built-in JSON parsing.

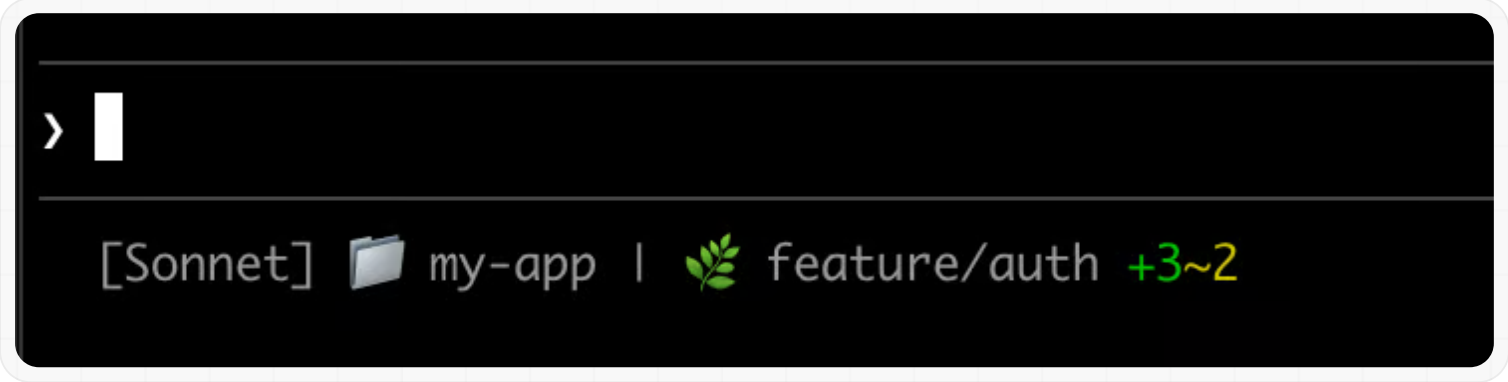
### Context window usage

Display the current model and context window usage with a visual progress bar. Each script reads JSON from stdin, extracts the `used_percentage` field, and builds a 10-character bar where filled blocks (▓) represent usage:

```
#!/bin/bash
# Read all of stdin into a variable
input=$(cat)

# Extract fields with jq, "// 0" provides fallback for null
MODEL=$(echo "$input" | jq -r '.model.display_name')
PCT=$(echo "$input" | jq -r '.context_window.used_percentage // 0' | cut -d. -
f1)

# Build progress bar: printf -v creates a run of spaces, then
# ${var// /▓} replaces each space with a block character
BAR_WIDTH=10
FILLED=$((PCT * BAR_WIDTH / 100))
EMPTY=$((BAR_WIDTH - FILLED))
BAR=""
[ "$FILLED" -gt 0 ] && printf -v FILL "%${FILLED}s" && BAR="${FILL// /▓}"
[ "$EMPTY" -gt 0 ] && printf -v PAD "%${EMPTY}s" && BAR="${BAR}${PAD// /░}"

echo "[$MODEL] $BAR $PCT%"
```

## Git status with colors

Show git branch with color-coded indicators for staged and modified files. This script uses **ANSI escape codes** for terminal colors: `\033[32m` is green, `\033[33m` is yellow, and `\033[0m` resets to default.



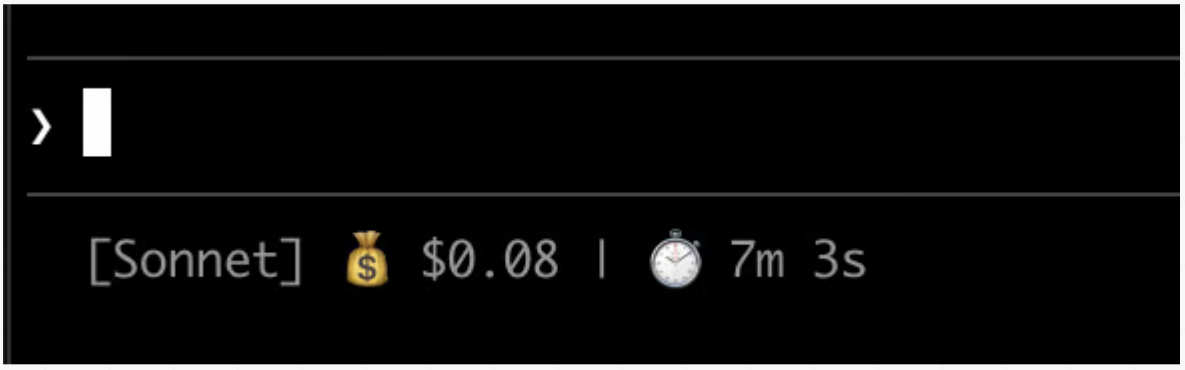

Each script checks if the current directory is a git repository, counts staged and modified files, and displays color-coded indicators:

```
#!/bin/bash
input=$(cat)

MODEL=$(echo "$input" | jq -r '.model.display_name')
DIR=$(echo "$input" | jq -r '.workspace.current_dir')

GREEN='\033[32m'
YELLOW='\033[33m'
RESET='\033[0m'

if git rev-parse --git-dir > /dev/null 2>&1; then
    BRANCH=$(git branch --show-current 2>/dev/null)
    STAGED=$(git diff --cached --numstat 2>/dev/null | wc -l | tr -d ' ')
    MODIFIED=$(git diff --numstat 2>/dev/null | wc -l | tr -d ' ')

    GIT_STATUS=""
    [ "$STAGED" -gt 0 ] && GIT_STATUS="${GREEN}+${STAGED}${RESET}"
    [ "$MODIFIED" -gt 0 ] &&
GIT_STATUS="${GIT_STATUS}${YELLOW}~${MODIFIED}${RESET}"

    echo -e "[$MODEL] 📁 ${DIR##*/} | 🌿 $BRANCH $GIT_STATUS"
else
    echo "[$MODEL] 📁 ${DIR##*/}"
fi
```

## Cost and duration tracking

Track your session's API costs and elapsed time. The `cost.total_cost_usd` field accumulates the estimated cost of all API calls in the current session. The `cost.total_duration_ms` field measures total elapsed time since the session started, while `cost.total_api_duration_ms` tracks only the time spent waiting for API responses.

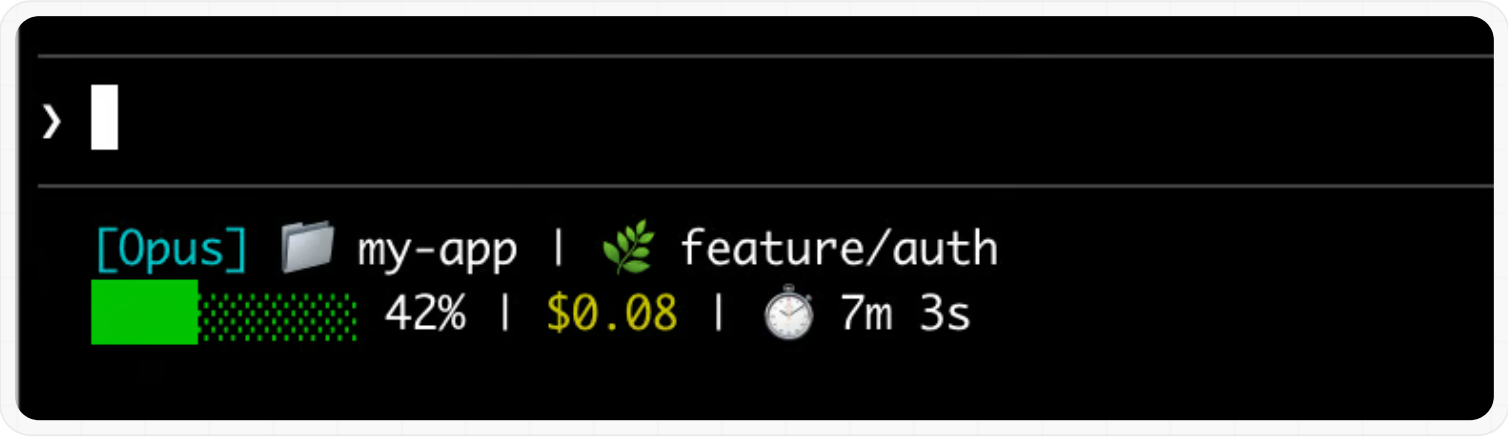
Each script formats cost as currency and converts milliseconds to minutes and seconds:



```
#!/bin/bash
input=$(cat)

MODEL=$(echo "$input" | jq -r '.model.display_name')
COST=$(echo "$input" | jq -r '.cost.total_cost_usd // 0')
DURATION_MS=$(echo "$input" | jq -r '.cost.total_duration_ms // 0')

COST_FMT=$(printf '$%.2f' "$COST")
DURATION_SEC=$((DURATION_MS / 1000))
MINS=$((DURATION_SEC / 60))
SECS=$((DURATION_SEC % 60))

echo "[$MODEL] 💰 $COST_FMT | ⏱️ ${MINS}m ${SECS}s"
```

## Display multiple lines

Your script can output multiple lines to create a richer display. Each `echo` statement produces a separate row in the status area.

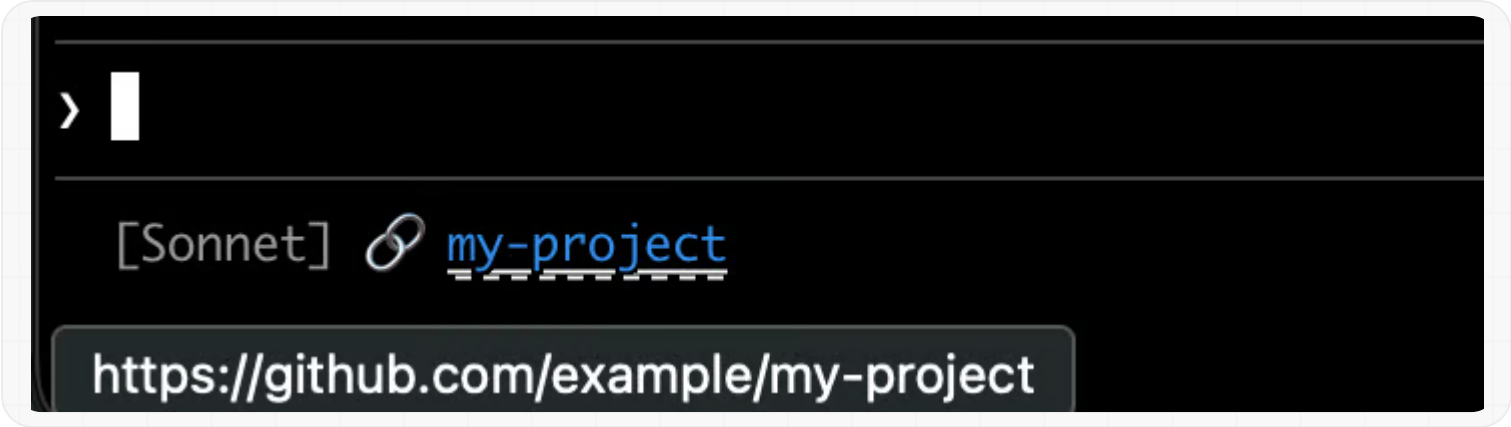



This example combines several techniques: threshold-based colors (green under 70%, yellow 70-89%, red 90%+), a progress bar, and git branch info. Each `print` or `echo` statement creates a separate row:

```
#!/bin/bash
input=$(cat)

MODEL=$(echo "$input" | jq -r '.model.display_name')
DIR=$(echo "$input" | jq -r '.workspace.current_dir')
COST=$(echo "$input" | jq -r '.cost.total_cost_usd // 0')
PCT=$(echo "$input" | jq -r '.context_window.used_percentage // 0' | cut -d. -
f1)
DURATION_MS=$(echo "$input" | jq -r '.cost.total_duration_ms // 0')

CYAN='\033[36m'; GREEN='\033[32m'; YELLOW='\033[33m'; RED='\033[31m';
RESET='\033[0m'

# Pick bar color based on context usage
if [ "$PCT" -ge 90 ]; then BAR_COLOR="$RED"
elif [ "$PCT" -ge 70 ]; then BAR_COLOR="$YELLOW"
else BAR_COLOR="$GREEN"; fi

FILLED=$((PCT / 10)); EMPTY=$((10 - FILLED))
printf -v FILL "%${FILLED}s"; printf -v PAD "%${EMPTY}s"
BAR="${FILL// /█}${PAD// /░}"

MINS=$((DURATION_MS / 60000)); SECS=$(((DURATION_MS % 60000) / 1000))

BRANCH=""
git rev-parse --git-dir > /dev/null 2>&1 && BRANCH=" | 🌿 $(git branch --show-
current 2>/dev/null)"

echo -e "${CYAN}[$MODEL]${RESET} 📂 ${DIR##*/}$BRANCH"
COST_FMT=$(printf '$%.2f' "$COST")
echo -e "${BAR_COLOR}${BAR}${RESET} ${PCT}% | ${YELLOW}${COST_FMT}${RESET} |
⏱️ ${MINS}m ${SECS}s"
```

## Clickable links

This example creates a clickable link to your GitHub repository. It reads the git remote URL, converts SSH format to HTTPS with `sed`, and wraps the repo name in OSC 8 escape codes. Hold Cmd (macOS) or Ctrl (Windows/Linux) and click to open the link in your browser.




Each script gets the git remote URL, converts SSH format to HTTPS, and wraps the repo name in OSC 8 escape codes. The Bash version uses `printf '%b'` which interprets backslash escapes more reliably than `echo -e` across different shells:

```
#!/bin/bash
input=$(cat)

MODEL=$(echo "$input" | jq -r '.model.display_name')

# Convert git SSH URL to HTTPS
REMOTE=$(git remote get-url origin 2>/dev/null | sed
's/git@github.com:/https:\/\/github.com\//' | sed 's/\.git$//')

if [ -n "$REMOTE" ]; then
    REPO_NAME=$(basename "$REMOTE")
    # OSC 8 format: \e]8;;URL\a then TEXT then \e]8;;\a
    # printf %b interprets escape sequences reliably across shells
    printf '%b' "[$MODEL] 🔗 \e]8;;${REMOTE}\a${REPO_NAME}\e]8;;\a\n"
else
    echo "[$MODEL]"
fi
```

## Rate limit usage

Display Claude.ai subscription rate limit usage in the status line. The `rate_limits` object contains `five_hour` (5-hour rolling window) and `seven_day` (weekly) windows. Each window provides `used_percentage` (0-100) and `resets_at` (Unix epoch seconds when the window resets).

This field is only present for Claude.ai subscribers (Pro/Max) after the first API response. Each script handles the absent field gracefully:

```
#!/bin/bash
input=$(cat)

MODEL=$(echo "$input" | jq -r '.model.display_name')
# "// empty" produces no output when rate_limits is absent
FIVE_H=$(echo "$input" | jq -r '.rate_limits.five_hour.used_percentage //
empty')
WEEK=$(echo "$input" | jq -r '.rate_limits.seven_day.used_percentage //
empty')

LIMITS=""
[ -n "$FIVE_H" ] && LIMITS="5h: $(printf '%.0f' "$FIVE_H")%"
[ -n "$WEEK" ] && LIMITS="${LIMITS:+$LIMITS }7d: $(printf '%.0f' "$WEEK")%"

[ -n "$LIMITS" ] && echo "[$MODEL] | $LIMITS" || echo "[$MODEL]"
```

## Cache expensive operations

Your status line script runs frequently during active sessions. Commands like `git status` or `git diff` can be slow, especially in large repositories. This example caches git information to a temp file and only refreshes it every 5 seconds.

The cache filename needs to be stable across status line invocations within a session, but unique across sessions so concurrent sessions in different repositories don't read each other's cached git state. Process-based identifiers like `$$`, `os.getpid()`, or `process.pid` change on every invocation and defeat the cache. Use the `session_id` from the JSON input instead: it's stable for the lifetime of a session and unique per session.

Each script checks if the cache file is missing or older than 5 seconds before running git commands:

```
#!/bin/bash
input=$(cat)

MODEL=$(echo "$input" | jq -r '.model.display_name')
DIR=$(echo "$input" | jq -r '.workspace.current_dir')
SESSION_ID=$(echo "$input" | jq -r '.session_id')

CACHE_FILE="/tmp/statusline-git-cache-$SESSION_ID"
CACHE_MAX_AGE=5  # seconds

cache_is_stale() {
    [ ! -f "$CACHE_FILE" ] || \
    # stat -f %m is macOS, stat -c %Y is Linux
    [ $(($(date +%s) - $(stat -f %m "$CACHE_FILE" 2>/dev/null || stat -c %Y
"$CACHE_FILE" 2>/dev/null || echo 0))) -gt $CACHE_MAX_AGE ]
}

if cache_is_stale; then
    if git rev-parse --git-dir > /dev/null 2>&1; then
        BRANCH=$(git branch --show-current 2>/dev/null)
        STAGED=$(git diff --cached --numstat 2>/dev/null | wc -l | tr -d ' ')
        MODIFIED=$(git diff --numstat 2>/dev/null | wc -l | tr -d ' ')
        echo "$BRANCH|$STAGED|$MODIFIED" > "$CACHE_FILE"
    else
        echo "||" > "$CACHE_FILE"
    fi
fi

IFS='|' read -r BRANCH STAGED MODIFIED < "$CACHE_FILE"

if [ -n "$BRANCH" ]; then
    echo "[$MODEL] 📁 ${DIR##*/} | 🌿 $BRANCH +$STAGED ~$MODIFIED"
else
    echo "[$MODEL] 📁 ${DIR##*/}"
fi
```

## Windows configuration

On Windows, Claude Code runs status line commands through Git Bash when Git Bash is installed, or through PowerShell when Git Bash is absent.

Git Bash treats unquoted backslashes as escape characters, so a Windows-style path such as

`C:\Users\username\script.mjs` reaches the script runner with its separators removed and the command fails without a visible error. Write file paths in the `command` string with forward slashes, as shown in the examples below. The `~` shorthand also works and expands to your Windows home directory.

To run a PowerShell script as your status line, invoke it via `powershell`. This works whether Claude Code routes the command through Git Bash or PowerShell:

```
{
  "statusLine": {
    "type": "command",
    "command": "powershell -NoProfile -File
C:/Users/username/.claude/statusline.ps1"
  }
}
```

Or, when Git Bash is installed, run a Bash script directly:

```
{
  "statusLine": {
    "type": "command",
    "command": "~/.claude/statusline.sh"
  }
}
```

## Subagent status lines

The `subagentStatusLine` setting renders a custom row body for each **subagent** shown in the agent panel below the prompt. Use it to replace the default `name · description · token count` row with your own formatting.

```
{
  "subagentStatusLine": {
    "type": "command",
    "command": "~/.claude/subagent-statusline.sh"
  }
}
```

The command runs once per refresh tick with all visible subagent rows passed as a single JSON object on stdin. The input includes the **base hook fields** plus `columns` (the usable row width) and a `tasks` array, where each task has `id`, `name`, `type`, `status`, `description`, `label`, `startTime`, `tokenCount`, `tokenSamples`, and `cwd`.

Write one JSON line to stdout per row you want to override, in the form `{"id": "<task id>", "content": "<row body>"}`. The `content` string is rendered as-is, including ANSI colors and OSC 8 hyperlinks. Omit a task's `id` to keep the default rendering for that row; emit an empty `content` string to hide it.

The same trust and `disableAllHooks` gates that apply to `statusLine` apply here. Plugins can ship a default `subagentStatusLine` in their `settings.json`.

## Tips

- **Test with mock input**: `echo '{"model":{"display_name":"Opus"},"workspace":{"current_dir":"/home/user/project"},"context_window":{"used_percentage":25},"session_id":"test-session-abc"}' | ./statusline.sh`
- **Keep output short**: the status bar has limited width, so long output may get truncated or wrap awkwardly
- **Cache slow operations**: your script runs frequently during active sessions, so commands like `git status` can cause lag. See the **caching example** for how to handle this.

Community projects like **ccstatusline** and **starship-claude** provide pre-built configurations with themes and additional features.

## Troubleshooting

### Status line not appearing

- Verify your script is executable: `chmod +x ~/.claude/statusline.sh`
- Check that your script outputs to stdout, not stderr
- Run your script manually to verify it produces output
- On Windows with Git Bash installed, backslashes in the `command` path are likely being consumed as escape characters before the script runs. Use forward slashes in the path. See **Windows configuration**.
- If `disableAllHooks` is set to `true` in your settings, the status line is also disabled. Remove this

setting or set it to `false` to re-enable.

- Run `claude --debug` to log the exit code and stderr from the first status line invocation in a session
- Ask Claude to read your settings file and execute the `statusLine` command directly to surface errors

**Status line shows `--` or empty values**

- Fields may be `null` before the first API response completes
- Handle null values in your script with fallbacks such as `// 0` in jq
- Restart Claude Code if values remain empty after multiple messages

**Context percentage shows unexpected values**

- Use `used_percentage` for the simplest accurate context state
- Context percentage may differ from `/context` output due to when each is calculated

**OSC 8 links not clickable**

- Verify your terminal supports OSC 8 hyperlinks (iTerm2, Kitty, WezTerm)
- Terminal.app does not support clickable links
- If link text appears but isn't clickable, Claude Code may not have detected hyperlink support in your terminal. This commonly affects Windows Terminal and other emulators not in the auto-detection list. Set the `FORCE_HYPERLINK` environment variable to override detection before launching Claude Code:

  ```
  FORCE_HYPERLINK=1 claude
  ```

  In PowerShell, set the variable in the current session first:

  ```
  $env:FORCE_HYPERLINK = "1"; claude
  ```

- SSH and tmux sessions may strip OSC sequences depending on configuration
- If escape sequences appear as literal text like `\e]8;;`, use `printf '%b'` instead of `echo -e` for more reliable escape handling

**Display glitches with escape sequences**

- Complex escape sequences (ANSI colors, OSC 8 links) can occasionally cause garbled output if they overlap with other UI updates
- If you see corrupted text, try simplifying your script to plain text output
- Multi-line status lines with escape codes are more prone to rendering issues than single-line plain text

**Workspace trust required**

- The status line command only runs if you've accepted the workspace trust dialog for the current directory. Because `statusLine` executes a shell command, it requires the same trust acceptance as hooks and other shell-executing settings.
- If trust isn't accepted, you'll see the notification `statusline skipped · restart to fix` instead of your status line output. Restart Claude Code and accept the trust prompt to enable it.

**Script errors or hangs**

- Scripts that exit with non-zero codes or produce no output cause the status line to go blank
- Slow scripts block the status line from updating until they complete. Keep scripts fast to avoid stale output.
- If a new update triggers while a slow script is running, the in-flight script is cancelled
- Test your script independently with mock input before configuring it

**Notifications share the status line row**

- System notifications like MCP server errors and auto-updates display on the right side of the same row as your status line. Transient notifications such as the context-low warning also cycle through this area.
- Enabling verbose mode adds a token counter to this area
- On narrow terminals, these notifications may truncate your status line output